DEVOPS SHACK ● devopsshack.com ● @devopsshack

DevOps CI/CD

SCENARIO-BASED INTERVIEW Q&A

GitHub Actions	GitLab CI/CD	Jenkins
Docker	Kubernetes	DevSecOps
Terraform	ArgoCD	AWS CodePipeline

Aditya Jaiswal
Principal DevOps Engineer & Educator

2026 Edition
16 Chapters · 100 Questions

TABLE OF CONTENTS

DevOps CI/CD Scenario-Based Interview Q&A

CH 01

CI/CD Fundamentals

Q 001

What is CI/CD and why is it critical in modern software delivery?

▶ ANSWER

CI/CD stands for Continuous Integration and Continuous Delivery/Deployment. It automates the building, testing, and deploying of applications so teams ship faster with fewer manual errors and faster feedback loops.

- Continuous Integration (CI) — merge code frequently; automated build and test triggered on every commit
- Continuous Delivery (CD) — code is always in a deployable state; a human approves the final production push
- Continuous Deployment — fully automated all the way to production without any human gate
- Reduces integration conflicts by merging small, incremental changes instead of week-long branches
- Accelerates the feedback loop: developers know within minutes if a change broke something
- Enables faster delivery of value to users — elite teams deploy multiple times per day

Pro Tip Think of CI/CD as an assembly line — each station validates quality before passing to the next. Broken parts never reach the customer.

Q 002

You joined a team deploying manually via SSH. How do you introduce CI/CD without disrupting current workflows?

▶ ANSWER

A phased approach minimises disruption. Start by adding visibility and automating safety nets before automating the actual deployment steps. Earn trust at each phase before moving to the next.

- Phase 1 — Git hygiene: enforce branching strategy (main, feature/, hotfix/) and PR review workflow
- Phase 2 — CI foundation: add GitHub Actions or GitLab CI for automated build and unit tests on every PR
- Phase 3 — Containerise: Dockerize all services, push images to a registry from CI on merge
- Phase 4 — Staging CD: auto-deploy to staging on every main branch merge; keep production manual
- Phase 5 — Production CD: add health checks, approval gates, auto-rollback, then enable full CD

- Throughout: document every pipeline decision, train the team, and celebrate each phase completion

> **Pro Tip** Never big-bang replace a working system. Each phase earns trust. Without trust, even perfect automation gets disabled after the first incident.

Q 003 What is the difference between Continuous Delivery and Continuous Deployment?

▶ ANSWER

Both extend Continuous Integration beyond build and test, but they differ in how far automation extends toward production.

- Continuous Delivery — every commit is production-ready; a human clicks deploy when ready
- Continuous Deployment — every passing build is automatically deployed to production without any human step
- Continuous Delivery is preferred in regulated industries: finance, healthcare, government — change approval is required
- Continuous Deployment suits high-velocity SaaS products needing many deploys per day
- You can mix strategies: Continuous Deployment to staging, Continuous Delivery to production
- Neither is superior — the right choice depends on your risk tolerance and business context

---

Q 004 Explain what pipeline-as-code means and why it matters.

▶ ANSWER

Pipeline-as-code means your CI/CD pipeline is defined in a configuration file (Jenkinsfile, .github/workflows/ci.yml, .gitlab-ci.yml) stored in version control alongside the application code it builds.

- Version controlled — every pipeline change is tracked, reviewable, and reversible with git revert
- Reproducible — any engineer can read the file and understand exactly how the pipeline works
- Auditable — git blame shows who changed the pipeline, when, and why (via commit message)
- Testable — pipeline YAML can be linted with actionlint, yamllint before it ever runs
- Collaborative — teams propose pipeline improvements via pull requests, same as any code change
- Disaster recovery — if your CI server disappears, the pipeline definition is safely in Git

Example

yaml / shell

```yaml
# .github/workflows/ci.yml — pipeline-as-code example
name: CI Pipeline
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install deps
        run: npm ci
      - name: Run tests
        run: npm test -- --coverage
      - name: Lint
        run: npm run lint
```

Q 005

A developer says: 'Why run tests in CI when I already tested locally?' How do you respond?

▶ ANSWER

Local environments are inconsistent, incomplete, and isolated to one person. CI provides a neutral, reproducible environment that represents the state of the entire team's code merged together.

- Local machines differ: OS version, Node/Java/Python version, missing env vars, different databases
- Eliminates 'works on my machine' — CI uses a locked, fresh environment built from scratch every run
- Catches integration issues: your code merged with Alice's and Bob's code may break in combination
- Enforces consistent toolchain versions — no surprises from a locally cached or upgraded package
- CI runs in isolation without leftover state (temp files, local DB rows, cached tokens)
- Provides a neutral quality gate: the team's code passes a shared bar, not just yours

Pro Tip CI is not about distrust — it is about the fact that code written by many people, when merged, can break in ways no individual's laptop can predict.

Q 006

What is a build artifact and how should it be managed in a CI/CD pipeline?

▶ ANSWER

A build artifact is any file produced by a build step — compiled binaries, Docker images, JAR files, npm packages, test reports. Proper artifact management ensures traceability and prevents rebuilding.

- Artifacts should be immutable once built — never modify an artifact after it passes tests
- Tag with git SHA for unique, traceable identification back to the exact commit
- Store in a registry or artifact repository: Docker images in GHCR/ECR, JARs in Nexus/Artifactory
- The same artifact that passed staging tests must be the one deployed to production — no rebuilding
- Retention policies: balance storage cost with rollback needs (e.g., keep 90 days of image tags)
- Artifact signing (cosign, Notation) adds provenance: verify the image was built by your pipeline

Q 007

Explain the concept of 'shift left' in DevOps and how CI/CD enables it.

▶ ANSWER

Shift left means moving quality, security, and compliance checks earlier in the development process — catching defects when they are cheapest and fastest to fix, not after deployment.

- Before CI/CD: security reviews happen at release time — months after the code was written
- With CI/CD: linting, SAST, secret scanning, and dependency checks run on every commit
- Cost of fixing a bug: $1 in development → $10 in CI → $100 in staging → $1000 in production
- Developer feedback is immediate — fix issues in the same context you introduced them
- Examples: IDE linting, pre-commit hooks, PR status checks, SAST in CI, DAST in staging
- Result: fewer issues reach production, and those that do are found and fixed faster

CH 02

GitHub Actions Deep Dive

Q 008

Describe the full structure of a GitHub Actions workflow file.

▶ ANSWER

A GitHub Actions workflow is a YAML file in .github/workflows/ consisting of triggers, permissions, environment variables, and jobs with sequential steps.

- name — human-readable workflow name displayed in the GitHub Actions UI
- on — trigger events: push, pull_request, schedule (cron), workflow_dispatch, workflow_call
- permissions — fine-grained GITHUB_TOKEN scopes; default is read-only since 2023
- env — global key-value environment variables available to all jobs in the workflow
- concurrency — prevent parallel runs of the same workflow on the same branch
- jobs — top-level units of work; run in parallel unless needs: creates a dependency
- steps — sequential tasks within a job; each step is either uses: (action) or run: (shell)

Example

yaml / shell

```yaml
name: Full Workflow Example
on:
  push:
    branches: [main]
    paths-ignore: ['**.md', 'docs/**']
  workflow_dispatch:
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
permissions:
  contents: read
  packages: write
env:
  REGISTRY: ghcr.io
jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
```

```
      image-tag: ${{ steps.tag.outputs.tag }}
    steps:
      - uses: actions/checkout@v4
      - id: tag
        run: echo "tag=${{ github.sha }}" >> $GITHUB_OUTPUT
```

Q 009

How do you use paths-ignore and paths filters to control workflow triggers precisely?

▶ ANSWER

Path filters let you run different workflows depending on which files actually changed in a push — avoiding unnecessary builds for documentation, config, or unrelated service changes.

- paths-ignore: workflow skips if ONLY excluded paths changed in the push
- paths: (opposite) — workflow only runs if at least one included path changed
- Both support glob patterns: **, *.md, src/**/*.ts, docs/
- Cannot use both paths and paths-ignore in the same trigger event
- For per-service filtering in a monorepo, use dorny/paths-filter action instead
- Note: paths filters do not apply to workflow_dispatch (manual triggers always run)

Example

yaml / shell

```yaml
on:
  push:
    branches: [main]
    paths-ignore:
      - '**.md'
      - 'docs/**'
      - '.gitignore'
      - 'release/**'
      - '*.txt'

# Only trigger on specific paths:
on:
  push:
    paths:
      - 'src/**'
      - 'package*.json'
```

```
      - 'Dockerfile'
```

Q 010

Explain GitHub Actions matrix strategy with a real-world microservices use case.

▶ ANSWER

Matrix strategy runs a job multiple times with different variable combinations simultaneously. This is the foundation of building only changed microservices in parallel.

- Define a matrix of variables under strategy.matrix — each combination becomes a parallel job
- fail-fast: false — allows other matrix jobs to complete even if one fails
- matrix.include — add extra variables to specific matrix combinations
- matrix.exclude — remove specific combinations without rewriting the entire matrix
- Dynamic matrices — build the JSON matrix in a prior job based on changed files
- fromJson() — convert the JSON string output from a prior job into a live matrix

Example

yaml / shell

```yaml
jobs:
  build:
    strategy:
      fail-fast: false
      matrix: ${{ fromJson(needs.detect.outputs.matrix) }}
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build ${{ matrix.service }}
        run: |
          docker build \
            -t ghcr.io/myorg/${{ matrix.service }}:${{ github.sha }} \
            ./src/${{ matrix.service }}
      - name: Push
        run: docker push ghcr.io/myorg/${{ matrix.service }}:${{ github.sha }}
```

Q 011

How do you securely manage and pass secrets in GitHub Actions workflows?

▶ ANSWER

GitHub provides an encrypted secrets store at repo, environment, and org level. Secrets are masked in logs and never exposed in workflow files.

- Store secrets at: repo Settings → Secrets and Variables → Actions
- Reference in YAML as ${{ secrets.SECRET_NAME }} — only available during the run
- Organization secrets: shared across multiple repos without duplication
- Environment secrets: scoped to a specific deployment environment (e.g., production)
- Secrets are masked in logs: even if you echo them, they appear as ***
- OIDC (best practice): eliminates static long-lived secrets for cloud provider authentication
- Never pass secrets as CLI arguments — they appear in process listings

---

Q 012

How do you share data between jobs in a GitHub Actions workflow?

▶ ANSWER

Jobs run on separate, isolated runners. There are three mechanisms to share data: job outputs for small values, artifacts for files, and cache for dependency restoration.

- Job outputs: set with echo 'key=value' >> $GITHUB_OUTPUT; read via needs.<job>.outputs.key
- Upload/download artifact: actions/upload-artifact and actions/download-artifact for files
- Cache (actions/cache): restore dependency directories across runs, not just jobs
- Artifacts persist for 90 days (configurable); cache expires after 7 days of no access
- For large binaries (Docker images), push to a registry and pull in the next job
- Job outputs are limited to strings; use artifacts for structured data or files

Example

yaml / shell

```yaml
jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      version: ${{ steps.ver.outputs.version }}
    steps:
      - id: ver
        run: echo "version=$(cat VERSION)" >> $GITHUB_OUTPUT
      - run: make build
```

```
      - uses: actions/upload-artifact@v4
        with:
          name: binaries
          path: dist/

  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/download-artifact@v4
        with: { name: binaries }
      - run: echo Deploying version ${{ needs.build.outputs.version }}
```

Q 013

What is OIDC authentication in GitHub Actions and why should you use it instead of static AWS keys?

▶ ANSWER

OIDC (OpenID Connect) allows GitHub Actions to request short-lived cloud credentials by presenting a cryptographically signed JWT — eliminating the need to store any long-lived secrets.

- GitHub issues a signed JWT token for each workflow run with claims: repo, branch, SHA, environment
- AWS, GCP, and Azure all support OIDC federation — they verify the JWT and issue temporary credentials
- Credentials expire after the workflow run — there is nothing to rotate or accidentally expose
- Trust policy scoped to specific repo, branch, or environment — least-privilege principle
- Full audit trail: cloud provider logs show exactly which repo and workflow assumed the role
- Setup once: create OIDC provider + IAM role; no secret rotation ever needed again

Example

yaml / shell

```
permissions:
  id-token: write   # Required for OIDC
  contents: read

steps:
  - uses: aws-actions/configure-aws-credentials@v4
    with:
      role-to-assume: ${{ secrets.AWS_ROLE_TO_ASSUME }}
      aws-region: ap-south-1
```

```
  - name: Verify identity
    run: aws sts get-caller-identity

  - name: Push to ECR
    run: |
      aws ecr get-login-password | \
      docker login --username AWS \
        --password-stdin ${{ env.ECR_REGISTRY }}
```

Q 014

How do you implement environment-specific deployment gates with approval in GitHub Actions?

▶ ANSWER

GitHub Environments add required reviewers, wait timers, and scoped secrets to deployment jobs — creating auditable, gated release processes without any third-party tools.

- Create environments at: repo Settings → Environments (e.g., staging, production)
- Required reviewers: named people or teams must approve before the job runs
- Wait timer: e.g., 15 minutes after PR merge before auto-proceeding to staging
- Protection rules: restrict which branches can deploy to this environment
- Every approval is recorded with: approver identity, timestamp, deployment SHA
- Environment secrets: DATABASE_URL, API_KEY scoped only to that environment's jobs

Example

yaml / shell

```
jobs:
  deploy-staging:
    environment: staging
    runs-on: ubuntu-latest
    steps:
      - run: kubectl apply -f release/ -n staging

  deploy-production:
    needs: deploy-staging
    environment:
      name: production
      url: https://app.devopsshack.com
```

```
    runs-on: ubuntu-latest
    steps:
      - run: kubectl apply -f release/ -n production
```

Q 015

How do you implement GitHub Actions reusable workflows for a platform engineering team managing 20+ service repos?

▶ ANSWER

Reusable workflows are called from multiple repos — define the pipeline once and all consumers inherit updates immediately, solving the copy-paste pipeline antipattern.

- Defined in a central platform repo with on: workflow_call trigger
- Called from any service repo with uses: org/platform/.github/workflows/build.yml@main
- Accepts inputs (service name, registry URL) and secrets forwarded from the caller
- A security fix in the reusable workflow propagates to all 20 repos automatically
- Pin callers to a version tag (@v1) to prevent breaking changes propagating immediately
- Composite actions are an alternative for sharing individual steps, not full workflows

Example

yaml / shell

```yaml
# platform repo: .github/workflows/docker-build.yml
on:
  workflow_call:
    inputs:
      service:
        required: true
        type: string
    secrets:
      GITHUB_TOKEN:
        required: true
jobs:
  build-push:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t ghcr.io/myorg/${{ inputs.service }}:${{
github.sha }} .
      - run: docker push ghcr.io/myorg/${{ inputs.service }}:${{ github.sha }}
```

```
# service repo: .github/workflows/ci.yml
jobs:
  build:
    uses: myorg/platform/.github/workflows/docker-build.yml@v1
    with:
      service: paymentservice
    secrets:
      GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
```

Q 016

How do you implement dependency caching in GitHub Actions to speed up Node.js, Python, and Go builds?

▶ ANSWER

Dependency caching stores downloaded packages between runs so subsequent builds skip the download step entirely. Each ecosystem has a recommended cache key pattern.

- Use actions/cache with a key based on the lockfile hash — only invalidates when dependencies change
- Node.js: cache key based on package-lock.json hash; restore npm ci from ~/.npm
- Python: cache key based on requirements.txt hash; restore pip from ~/.cache/pip
- Go: cache key based on go.sum hash; restore from ~/go/pkg/mod
- restore-keys: fallback to a partial match if exact hash is not found
- actions/setup-node, setup-python, setup-go all have built-in cache: true parameter — use it

Example

yaml / shell

```
- uses: actions/setup-node@v4
  with:
    node-version: '20'
    cache: 'npm'   # built-in npm cache

# Or manual cache for advanced control:
- uses: actions/cache@v4
  with:
    path: ~/.npm
    key: ${{ runner.os }}-npm-${{ hashFiles('**/package-lock.json') }}
    restore-keys: |
      ${{ runner.os }}-npm-

- run: npm ci   # uses cache if available
```

CH 03

Docker & Container Workflows in CI/CD

Q 017

How do you optimise Docker image builds in CI/CD to minimise build time?

▶ ANSWER

Docker build optimisation is almost entirely about cache hit rate. Order layers by change frequency, use multi-stage builds, and leverage GitHub Actions cache for Docker layer persistence across runs.

- Copy dependency manifests first (package.json, pom.xml, requirements.txt), then source code
- This caches the expensive install step — only reruns when dependencies actually change
- Multi-stage builds: build in a full SDK image; copy only artifacts to a minimal runtime image
- Use .dockerignore to exclude node_modules, .git, test files from the build context
- Use BuildKit (DOCKER_BUILDKIT=1) for parallel stage execution and advanced caching
- GitHub Actions: use docker/build-push-action with cache-from and cache-to for cross-run caching
- Result: 8 minute builds can drop to under 90 seconds with proper cache configuration

Example

yaml / shell

```
# Optimised multi-stage Dockerfile
FROM node:20-alpine AS deps
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production

FROM node:20-alpine AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npm run build

FROM node:20-alpine AS runtime
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=deps  /app/node_modules ./node_modules
USER node
```

```
EXPOSE 3000
CMD ["node", "dist/index.js"]
```

Q 018

Your pipeline fails with 'permission_denied: write_package' pushing to GHCR. Walk through all possible fixes.

▶ ANSWER

GHCR push failures have four distinct root causes. Systematically check each one.

- Fix 1 — Add packages: write at the job level, not just the workflow level
- Fix 2 — Login with github.repository_owner, NOT github.actor (bot triggers cause actor mismatch)
- Fix 3 — First-time push: package may not exist yet; push manually once with a PAT to initialise
- Fix 4 — Org package settings: the org may restrict which repos can publish to the registry
- Fix 5 — Link the package to the repository in GitHub package settings
- Fix 6 — Verify GITHUB_TOKEN is not overridden by a workflow-level env GITHUB_TOKEN variable

Example

yaml / shell

```
# Job-level permissions are required
service-pipeline:
  runs-on: ubuntu-latest
  permissions:
    contents: read
    packages: write   # MUST be at job level

  steps:
    - name: Login to GHCR
      run: |
        echo "${{ secrets.GITHUB_TOKEN }}" | \
        docker login ghcr.io \
          -u ${{ github.repository_owner }} \
          --password-stdin

    - name: Build and push
      run: |
        docker build -t ghcr.io/${{ github.repository_owner }}/app:${{
github.sha }} .
        docker push ghcr.io/${{ github.repository_owner }}/app:${{ github.sha
}}
```

Pro Tip github.actor is whoever triggered the workflow — could be a bot. github.repository_owner is always the repo owner. Use repository_owner for GHCR login.

Q 019

How do you implement Docker image vulnerability scanning in a CI/CD pipeline?

ANSWER

Scan at both filesystem level before build (source code CVEs) and image level after build (OS package CVEs). Use exit codes to gate the pipeline progressively.

- trivy fs ./src/service — scans source code and dependencies before the Docker build
- trivy image myapp:sha — scans the built image's OS packages and installed libraries
- --exit-code 0 during adoption phase: generates reports without blocking pipeline
- --exit-code 1 in mature state: CRITICAL/HIGH findings fail the pipeline
- SARIF output: upload to GitHub Security tab for team-wide visibility and tracking
- Cache the Trivy vulnerability database: saves 30 seconds per scan via actions/cache

Example

yaml / shell

```yaml
- name: Install Trivy
  uses: aquasecurity/setup-trivy@v0.2.6
  with:
    version: latest
    cache: true

- name: Trivy filesystem scan
  run: |
    trivy fs ./src/${{ matrix.service }} \
      --severity HIGH,CRITICAL --exit-code 0

- name: Trivy image scan
  run: |
    trivy image ghcr.io/myorg/${{ matrix.service }}:${{ github.sha }} \
      --severity HIGH,CRITICAL \
      --exit-code 0 \
      --format sarif -o trivy.sarif
```

```
- uses: github/codeql-action/upload-sarif@v3
  with:
    sarif_file: trivy.sarif
```

Q 020

What Docker image tagging strategy should a production CI/CD pipeline use?

▶ ANSWER

A robust tagging strategy balances immutability for traceability with human-readable tags for operations. Never rely on :latest as the only tag.

- Git SHA tag — immutable, uniquely links an image to the exact commit that built it
- Semantic version tag (v1.2.3) — for release artifacts; created at tag push event
- :latest — only pushed on main branch builds; useful for dev/test environments, never prod
- Environment tag (staging, prod) — pair with SHA for environment-based pulls
- Multi-tag pattern: build once, push SHA + version + latest simultaneously
- Never overwrite an existing SHA tag — if you do, you break rollback traceability

Example

yaml / shell

```
- name: Build and multi-tag
  run: |
    IMAGE=ghcr.io/${{ github.repository_owner }}/app
    SHA=${{ github.sha }}
    VERSION=$(cat VERSION)

    docker build \
      -t $IMAGE:$SHA \
      -t $IMAGE:v$VERSION \
      -t $IMAGE:latest \
      .

    docker push $IMAGE:$SHA
    docker push $IMAGE:v$VERSION
    docker push $IMAGE:latest
```

Q 021

How do you handle secrets inside Docker containers securely during CI/CD?

▶ ANSWER

Secrets must never be embedded in image layers. Use BuildKit secret mounts for build-time secrets and runtime injection via orchestrator for application secrets.

- BuildKit --mount=type=secret: mounts a secret at build time without writing it to any layer
- Never use ARG or ENV for secrets — they appear in docker history and image metadata
- Runtime secrets: inject via K8s Secrets, ECS task definition environment, or Vault agent
- AWS Secrets Manager / GCP Secret Manager: application fetches secrets on startup
- Scan built images with Gitleaks or truffleHog to detect accidentally embedded secrets
- Multi-stage builds: secrets used in builder stage are not present in the final runtime image

Example

yaml / shell

```
# BuildKit secret mount — secret never in any image layer
# syntax=docker/dockerfile:1
FROM python:3.12-slim AS builder
RUN --mount=type=secret,id=pip_token \
    PIP_INDEX_URL=$(cat /run/secrets/pip_token) \
    pip install --no-cache-dir -r requirements.txt

FROM python:3.12-slim
COPY --from=builder /usr/local/lib/python3.12 /usr/local/lib/python3.12

# Build command:
# docker build --secret id=pip_token,env=PIP_TOKEN .
```

Q 022

How do you implement Docker layer caching in GitHub Actions across pipeline runs?

▶ ANSWER

GitHub Actions runners start fresh for every run. Without explicit cache configuration, Docker rebuilds every layer from scratch. Two approaches work: GitHub cache backend and registry cache.

- docker/build-push-action with cache-from: type=gha and cache-to: type=gha,mode=max
- Registry cache: cache-from: type=registry,ref=ghcr.io/org/app:cache-latest
- GitHub cache has 10 GB limit per repo — monitor usage and evict stale caches
- mode=max caches every stage (including intermediate stages in multi-stage builds)
- For self-hosted runners with shared disk: local type cache is fastest
- Verify cache hit: docker/build-push-action outputs cache-hit field to check

Example

```yaml / shell
- name: Build and push with cache
  uses: docker/build-push-action@v5
  with:
    context: ./src/${{ matrix.service }}
    push: true
    tags: ghcr.io/myorg/${{ matrix.service }}:${{ github.sha }}
    cache-from: type=gha,scope=${{ matrix.service }}
    cache-to: type=gha,scope=${{ matrix.service }},mode=max
```

Q 023

How do you implement a Docker image cleanup strategy in CI/CD to avoid registry storage costs?

▶ ANSWER

Container registries accumulate images rapidly in active CI/CD environments. Automated cleanup retains recent and tagged images while removing old untagged builds.

- GHCR: use gh-actions-ghcr-cleanup or GitHub's built-in retention policies for untagged images
- ECR: lifecycle policies automatically expire untagged images older than N days
- Retain: all semver-tagged images (v1.2.3), latest, and last 10 SHA tags
- Delete: untagged images older than 7 days, SHA tags older than 30 days
- Pipeline cleanup step: docker image prune -af after each build to free runner disk
- Registry audit: monthly report of image count and size per service to track growth

Example

```yaml / shell
# ECR lifecycle policy (JSON)
{
  "rules": [
```

```
      {
        "rulePriority": 1,
        "description": "Remove untagged images after 7 days",
        "selection": {
          "tagStatus": "untagged",
          "countType": "sinceImagePushed",
          "countUnit": "days",
          "countNumber": 7
        },
        "action": { "type": "expire" }
      }
    ]
  }
```

CH 04

Kubernetes Deployments via CI/CD

Q 024

How do you authenticate GitHub Actions to an AWS EKS cluster using OIDC without static credentials?

▶ ANSWER

OIDC federation eliminates long-lived AWS credentials. GitHub Actions exchanges a JWT for short-lived AWS role credentials, then generates a kubeconfig for kubectl.

- Create an OIDC provider in AWS IAM pointing to token.actions.githubusercontent.com
- Create an IAM role with trust policy conditioned on repo, branch, and/or environment claims
- Attach EKS permissions: eks:DescribeCluster minimum; plus K8s RBAC binding in the cluster
- Add id-token: write permission to the workflow job — required to request the JWT
- Use aws-actions/configure-aws-credentials@v4 with role-to-assume for credential exchange
- Run aws eks update-kubeconfig to write kubeconfig for kubectl commands

Example

yaml / shell

```yaml
permissions:
  id-token: write
  contents: read

steps:
  - uses: aws-actions/configure-aws-credentials@v4
    with:
      role-to-assume: ${{ secrets.AWS_ROLE_TO_ASSUME }}
      aws-region: ap-south-1

  - name: Update kubeconfig
    run: |
      aws eks update-kubeconfig \
        --region ap-south-1 \
        --name my-cluster

  - name: Verify access
    run: |
```

```
kubectl auth can-i get pods -n project
kubectl auth can-i update deployments -n project
```

Q 025

Your Kubernetes deployment is not picking up the new image after CI/CD completes. How do you diagnose and fix it?

▶ ANSWER

This is always caused by a manifest, image pull policy, or deployment sequencing issue. Work through a structured checklist.

- Check 1 — Verify the manifest YAML was updated with the new SHA before kubectl apply ran
- Check 2 — Confirm the updated manifest was committed and pushed to Git
- Check 3 — imagePullPolicy: IfNotPresent skips re-pulling if the tag already exists on the node
- Check 4 — Did kubectl apply run against the latest committed file or a stale checkout?
- Check 5 — kubectl describe deployment <n> shows the image field currently active
- Root fix — always use immutable SHA tags; :latest + IfNotPresent is the classic trap

Example

yaml / shell

```
# Check active image
kubectl get deployment myapp -n project \
  -o jsonpath='{.spec.template.spec.containers[0].image}'

# Force re-pull and rollout
kubectl rollout restart deployment/myapp -n project

# Watch rollout progress
kubectl rollout status deployment/myapp -n project
```

Q 026

How do you implement zero-downtime rolling deployments in Kubernetes through CI/CD?

▶ ANSWER

Kubernetes Rolling Updates provide zero-downtime natively. The key is configuring maxUnavailable, readiness probes, and graceful termination correctly.

- maxUnavailable: 0 — Kubernetes never terminates an old pod before a new one is ready and healthy
- maxSurge: 1 — allows one extra pod above the desired count during the rollout transition
- readinessProbe — K8s only routes traffic to a pod once this probe returns 200
- livenessProbe — auto-restarts a pod that becomes unhealthy after a successful startup
- terminationGracePeriodSeconds: 30 — gives in-flight requests time to drain before pod dies
- preStop hook: sleep 5 before SIGTERM gives the load balancer time to deregister the pod

Example

yaml / shell

```yaml
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
  template:
    spec:
      terminationGracePeriodSeconds: 30
      containers:
      - name: app
        image: ghcr.io/myorg/app:abc1234
        lifecycle:
          preStop:
            exec:
              command: ["sleep", "5"]
        readinessProbe:
          httpGet: { path: /health, port: 8080 }
          initialDelaySeconds: 10
          periodSeconds: 5
          failureThreshold: 3
```

Q 027

How do you verify a Kubernetes rollout succeeded in the pipeline and auto-rollback on failure?

▶ ANSWER

kubectl rollout status blocks the pipeline and exits non-zero on timeout. Wrap it with a health check and rollback command for full automation.

- kubectl rollout status blocks until all pods are running and ready or timeout hits
- --timeout=300s prevents the pipeline from hanging indefinitely on a stuck rollout
- Capture exit code: if status fails, immediately trigger kubectl rollout undo
- Follow up with a smoke test: curl the app health endpoint to confirm end-to-end
- Notify Slack with rollback details — service, SHA, link to failed pipeline run
- Post-rollback: the old ReplicaSet is kept by Kubernetes for fast re-rollback if needed

Example

yaml / shell

```yaml
- name: Verify rollout and auto-rollback
  run: |
    SERVICES=(adservice cartservice frontend paymentservice)
    NAMESPACE=project
    for svc in "${SERVICES[@]}"; do
      echo "==> Checking $svc..."
      if ! kubectl rollout status deployment/$svc \
           -n $NAMESPACE --timeout=300s; then
        echo "FAILED: Rolling back $svc"
        kubectl rollout undo deployment/$svc -n $NAMESPACE
        exit 1
      fi
    done
    echo '==> All deployments healthy'
```

Q 028 How do you manage Kubernetes namespaces and RBAC across dev, staging, and production environments in CI/CD?

▶ ANSWER

Separate namespaces per environment with environment-scoped RBAC ensure CI/CD service accounts have least-privilege access and changes cannot accidentally target the wrong environment.

- dev namespace: deployed on feature branch merges; full automation, no approval
- staging namespace: deployed on main branch merge; automated post-merge
- production namespace: protected with GitHub Environment approval gate
- CI service account is namespace-scoped, not cluster-admin — only verbs needed for deployments
- Use kubectl auth can-i to verify permissions before attempting deployments
- Separate kubeconfig contexts per environment stored as GitHub environment secrets

Q 029

Explain how you would implement blue-green deployment natively in Kubernetes without Argo Rollouts.

▶ ANSWER

Blue-green requires two identical Deployments running simultaneously with a Service selector switch as the traffic control mechanism — pure Kubernetes primitives, no extra tools.

- Deploy blue (current) Deployment with label: version: blue
- Service selector: version: blue — all traffic goes to blue
- Deploy green (new) Deployment with label: version: green; wait for all pods ready
- kubectl patch service to switch selector to version: green — instantaneous cutover
- Monitor error rate for 5–10 minutes; if healthy, delete blue Deployment
- Rollback: kubectl patch service back to version: blue — sub-second, zero-downtime

Example

yaml / shell

```
# Switch traffic to green (instant cutover)
kubectl patch service myapp -n project \
  -p '{"spec":{"selector":{"version":"green"}}}'

# Verify traffic is going to green
kubectl describe service myapp -n project | grep Selector

# Instant rollback to blue
kubectl patch service myapp -n project \
  -p '{"spec":{"selector":{"version":"blue"}}}'

# Cleanup blue after successful green
kubectl delete deployment myapp-blue -n project
```

CH 05

DevSecOps — Security in CI/CD

Q 030

What is DevSecOps and how does it fundamentally change the security model?

▶ ANSWER

DevSecOps integrates security into every stage of the CI/CD pipeline, making it a shared team responsibility from first commit to production — not a final gating checkpoint.

- Traditional security: separate security team reviews before release — slow, adversarial, expensive
- DevSecOps: security checks run automatically in CI on every single commit — fast, collaborative
- Shift left: catching a vulnerability at commit time costs $1; in production it costs $1000+
- Developers receive immediate, contextual security feedback — no waiting weeks for a report
- Key stages: pre-commit hooks → SAST in CI → SCA → image scan → IaC scan → DAST in staging
- Security becomes a quality attribute, not a separate phase — same as unit testing

Q 031

How do you implement secret scanning to prevent credentials from being committed to Git?

▶ ANSWER

Secret scanning should exist at two levels: local pre-commit hooks to block commits, and CI pipeline scanning as the last line of defence for anything that slips through.

- Gitleaks: scans Git history and staged files for 100+ secret patterns — API keys, JWT tokens, passwords
- Pre-commit hook with gitleaks: developer is blocked before the commit even happens
- CI pipeline: Gitleaks runs as the first job — fail the pipeline immediately if secrets are found
- --exit-code 1 in production pipelines: a detected secret must block the build
- GitHub Advanced Security: built-in secret scanning with automatic PR alerts
- Remediation: detected secrets must be revoked immediately, then removed from Git history

Example

yaml / shell

```
gitleaks:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
      with:
        fetch-depth: 0    # scan full history
    - name: Run Gitleaks
      run: |
        docker run --rm -v "$PWD:/repo" \
          zricethezav/gitleaks:latest detect \
          --source=/repo \
          --report-format json \
          --report-path /repo/gitleaks-report.json \
          --exit-code 1

    - uses: actions/upload-artifact@v4
      if: always()
      with:
        name: gitleaks-report
        path: gitleaks-report.json
```

Q 032

How do you add SonarQube static analysis (SAST) to a GitHub Actions CI pipeline?

▶ ANSWER

SonarQube SAST analyses source code for bugs, vulnerabilities, and code smells without running the application. Integrate it as a required status check on all pull requests.

- SonarQube Community can be self-hosted; SonarCloud is the managed SaaS option
- Runs sonar-scanner or Maven/Gradle sonar:sonar goal — language auto-detected
- Quality Gate: SonarQube evaluates results against a pass/fail threshold
- If Quality Gate fails: pipeline fails and the PR cannot be merged
- Metrics tracked: code coverage delta, new bugs, new vulnerabilities, code duplication
- Configure sonar.projectKey, sonar.sources, and sonar.exclusions in sonar-project.properties

Example

yaml / shell

```
- name: SonarQube Analysis
```

```
    env:
      SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
      SONAR_HOST_URL: ${{ secrets.SONAR_HOST_URL }}
    run: |
      docker run --rm \
        -e SONAR_TOKEN=$SONAR_TOKEN \
        -e SONAR_HOST_URL=$SONAR_HOST_URL \
        -v "$PWD:/usr/src" \
        sonarsource/sonar-scanner-cli \
        -Dsonar.projectKey=myapp \
        -Dsonar.sources=src \
        -Dsonar.qualitygate.wait=true
```

Q 033

What is IaC security scanning and how do you add Checkov to a CI/CD pipeline?

▶ ANSWER

IaC scanning analyses Terraform, CloudFormation, Kubernetes manifests, and Helm charts for security misconfigurations before they are ever applied to real infrastructure.

- Checkov: 2000+ built-in policies covering Terraform, CloudFormation, K8s, Helm, ARM, Bicep
- Detects: S3 buckets without encryption, security groups open to 0.0.0.0/0, pods running as root
- --soft-fail during adoption: report findings without blocking the pipeline
- Tighten over time: add --check CKV_AWS_18,CKV_K8S_30 to block specific critical policies
- Custom policies: write Python or YAML policies for organisation-specific rules
- Combine with tfsec, kube-score, and OPA/Conftest for layered IaC security

Example

yaml / shell

```
checkov:
  needs: gitleaks
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Run Checkov on Terraform
      run: |
        docker run --rm -v "$PWD:/tf" \
          bridgecrew/checkov \
          -d /tf \
```

```
            --framework terraform \
            --output cli \
            --output junitxml \
            --output-file-path . \
            --soft-fail
      - name: Run Checkov on Kubernetes
        run: |
          docker run --rm -v "$PWD:/tf" \
            bridgecrew/checkov \
            -d /tf/release \
            --framework kubernetes --soft-fail
```

Q 034

Explain DAST (Dynamic Application Security Testing) and how to integrate OWASP ZAP in CI/CD.

▶ ANSWER

DAST tests a running application from the outside, simulating real attacker behaviour. It finds vulnerabilities that SAST cannot: runtime misconfigurations, injection flaws, authentication gaps.

- DAST runs after the application is deployed to a staging environment
- OWASP ZAP Baseline Scan: passive scanning, completes in 2–5 minutes — good for every PR
- OWASP ZAP Full Scan: active scanning with attack payloads — 30+ minutes, weekly or pre-release
- ZAP runs as a Docker container pointed at the staging URL
- Generates HTML, JSON, and XML reports published as pipeline artifacts
- Combine SAST (SonarQube) + SCA (Trivy) + DAST (ZAP) for complete application security coverage

Example

yaml / shell

```
- name: OWASP ZAP Baseline Scan
  run: |
    docker run --rm \
      -v ${{ github.workspace }}/zap:/zap/wrk \
      ghcr.io/zaproxy/zaproxy:stable \
      zap-baseline.py \
      -t http://staging.myapp.com \
      -r zap-report.html \
      -x zap-report.xml \
      -I   # Don't fail on warnings
```

```
- uses: actions/upload-artifact@v4
  if: always()
  with:
    name: zap-security-report
    path: zap/zap-report.html
```

Q 035

What is an SBOM (Software Bill of Materials) and why has it become mandatory in DevSecOps?

▶ ANSWER

An SBOM is a formal, machine-readable inventory of every component in a software artifact. It is the foundation of modern supply chain security and regulatory compliance.

- Lists every direct and transitive dependency: name, version, licence, source location
- Enables instant CVE blast-radius assessment — 'which services use log4j 2.14.1?'
- SPDX and CycloneDX: two open standards for SBOM format (JSON or XML)
- Syft: open-source SBOM generator for images, filesystems, and source code
- Grype: vulnerability scanner that uses SBOMs as input for fast CVE matching
- US Executive Order 14028 (2021): mandates SBOMs for all software sold to US federal government

Example

yaml / shell

```
- name: Generate SBOM with Syft
  run: |
    docker run --rm \
      -v /var/run/docker.sock:/var/run/docker.sock \
      anchore/syft \
      ghcr.io/myorg/app:${{ github.sha }} \
      -o spdx-json > sbom.spdx.json

- name: Scan SBOM with Grype
  run: |
    docker run --rm \
      -v $PWD:/work anchore/grype \
      sbom:/work/sbom.spdx.json \
      --fail-on critical
```

```
  - uses: actions/upload-artifact@v4
    with:
      name: sbom-${{ github.sha }}
      path: sbom.spdx.json
```

CH 06

Pipeline Design Patterns

Q 036

How do you design a CI/CD pipeline for a monorepo with 12 microservices that builds only what changed?

▶ ANSWER

Change detection with dynamic matrix strategy is the definitive pattern. Build only the services whose files changed in the push — reducing typical build time by 80%.

- Use dorny/paths-filter action to detect which service directories changed
- Build a JSON matrix of changed services in a shell script published as a job output
- Shared code changes (protos/, libs/) trigger all services that depend on them
- strategy.matrix: ${{ fromJson(needs.detect.outputs.matrix) }} consumes the dynamic list
- fail-fast: false — one failing service does not block other services from building
- Single update-manifest job runs after all service builds complete, updating all changed tags

Example

yaml / shell

```
detect-changes:
  outputs:
    matrix: ${{ steps.set-matrix.outputs.matrix }}
  steps:
    - uses: dorny/paths-filter@v3
      id: filter
      with:
        filters: |
          adservice: ['src/adservice/**', 'protos/**']
          frontend:  ['src/frontend/**', 'protos/**']
    - id: set-matrix
      run: |
        SERVICES=()
        [[ '${{ steps.filter.outputs.adservice }}' == 'true' ]] \
          && SERVICES+='"adservice"'
        [[ '${{ steps.filter.outputs.frontend }}' == 'true' ]] \
          && SERVICES+='"frontend"'
        echo "matrix={\"service\":[$(IFS=,; echo ${SERVICES[*]})]}" >>
$GITHUB_OUTPUT
```

Q 037

What is GitOps and how does it fundamentally change the deployment model?

▶ ANSWER

GitOps makes Git the single source of truth for both application code and desired infrastructure state. A pull-based operator reconciles the cluster to match Git continuously.

- Declarative: desired cluster state defined as YAML files committed to a Git repository
- Pull-based: GitOps operator (ArgoCD, Flux) pulls changes from Git — CI never pushes directly
- CI still owns build: builds image, pushes to registry, updates manifest SHA in Git
- Operator owns deploy: detects manifest change, syncs cluster to match Git state
- Drift detection: operator continuously checks and alerts if live state diverges from Git
- Rollback = git revert: no special tooling needed — git history is the deployment history

Q 038

How do you implement a promotion pipeline that deploys to dev → staging → production safely?

▶ ANSWER

A promotion pipeline ensures the exact same artifact flows through environments without rebuilding. Each environment adds a quality gate before promotion.

- Dev: deploy on every feature branch commit for developer testing
- Staging: deploy automatically on main branch merge after CI tests pass
- Staging tests: automated integration + E2E tests run against the staging deployment
- Production gate: GitHub Environment with required reviewers blocks the job
- Production: kubectl apply the same manifest SHA that passed staging — no rebuild ever
- Immutable artifact: the image that passed staging is the image that goes to production

Q 039

Your pipeline takes 45 minutes to run. Walk through a systematic optimisation approach.

▶ ANSWER

Pipeline performance is a performance engineering problem. Measure first, identify the critical path, then apply targeted optimisations.

- Step 1: add timestamps and draw the job dependency graph to find the critical path
- Step 2: move independent jobs off the critical path to run in parallel
- Step 3: cache npm/Maven/pip/Go module directories between runs
- Step 4: enable Docker layer cache — biggest single win, can save 6–8 minutes alone
- Step 5: build only changed services (path filtering in monorepo)
- Step 6: run E2E and integration tests only on main branch, not every PR commit
- Step 7: use self-hosted runners for heavy build workloads to avoid shared resource contention

Pro Tip Docker layer cache + dependency caching alone typically cuts pipeline time from 40 minutes to under 10 minutes. Start there before optimising anything else.

Q 040

How do you handle database migrations safely in a CI/CD pipeline?

▶ ANSWER

Database migrations and application deployments must be decoupled. The schema must be backward-compatible with both old and new application code throughout the transition.

- Run migration job before deploying new application code — never simultaneously
- Backward-compatible schema: add nullable columns first; add NOT NULL constraint in a later deploy
- Versioned migrations: use Flyway, Liquibase, or Alembic to track and apply migrations in order
- Test on staging database before production — staging schema must mirror production
- Idempotent migrations: safe to run multiple times without side effects
- Maintain rollback scripts for every forward migration; test them in staging regularly

---

Q 041

What is the 'deploy-by-default, feature-flag new behaviour' pattern and when is it preferable to branch-per-feature?

▶ ANSWER

Feature flags decouple deployment from release. Code is deployed to production disabled, then gradually enabled for specific users or percentages — eliminating long-lived feature branches.

- Code is deployed with new behaviour behind a flag: if (flags.enabled('new-checkout')) {...}
- Flag is off by default in production — zero user impact at deploy time
- Progressive rollout: enable for internal users → 1% → 10% → 100%
- Instant kill switch: disable the flag if issues emerge, without a redeployment
- Eliminates merge conflicts from long-lived feature branches diverging for weeks
- Tools: LaunchDarkly, Split.io, Unleash (self-hosted), or simple database-backed flags

Q 042

How do you implement a pipeline-wide notification and reporting strategy for a DevOps team?

▶ ANSWER

A good notification strategy provides signal without noise — the right people get the right alerts with enough context to act immediately, without being flooded by non-actionable messages.

- Failure notifications: always sent to the team Slack channel with service, SHA, author, and run link
- Production deploy success: posted to #deployments channel with version and environment
- Security scan findings: posted to #security-alerts with severity and CVE details
- Weekly digest: DORA metrics summary (deployment frequency, MTTR, change failure rate)
- PagerDuty integration: production deployment failure or rollback triggers on-call alert
- Avoid notification fatigue: only alert on actionable events — not every PR CI run

CH 07

GitLab CI/CD

Q 043

Compare GitLab CI/CD pipeline structure with GitHub Actions.

▶ ANSWER

Both are YAML pipeline-as-code systems but use different structural primitives, terminology, and built-in features.

- GitLab: stages define execution order; jobs within the same stage run in parallel
- GitHub: jobs are defined directly; needs: creates explicit dependency edges
- GitLab: single .gitlab-ci.yml at repo root; GitHub: multiple .github/workflows/*.yml files
- GitLab: built-in container registry, package registry, environments, and review apps
- GitLab: variables at group, project, and environment scope with fine-grained access control
- GitLab: includes let you split pipeline into multiple files and import from other projects

Example

yaml / shell

```
# .gitlab-ci.yml
stages:
  - security
  - build
  - test
  - deploy

gitleaks:
  stage: security
  script:
    - docker run --rm -v "$PWD:/repo" zricethezav/gitleaks:latest detect --
source=/repo

build-image:
  stage: build
  script:
    - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA .
    - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
```

```
deploy-prod:
  stage: deploy
  environment: production
  when: manual
  script:
    - kubectl apply -f k8s/ -n production
  only:
    - main
```

Q 044

How do GitLab CI/CD variables work across group, project, and environment levels?

▶ ANSWER

GitLab variables are a hierarchical key-value system. Lower-level definitions override higher-level ones, enabling flexible multi-environment configuration without duplication.

- Group-level variables: inherited by all projects in the group — great for shared org secrets
- Project-level variables: override group variables for that specific project
- Environment-scoped variables: same key, different values per environment (DATABASE_URL in dev vs prod)
- Protected variables: only available in pipelines on protected branches (main, release/*)
- Masked variables: value hidden from job logs — use for secrets
- File type: value written to a temp file; path passed as env var — ideal for certificates and kubeconfigs

Q 045

How do you use GitLab Review Apps to create per-merge-request preview environments?

▶ ANSWER

Review Apps automatically deploy each merge request to a unique URL, letting reviewers test changes live before merging. They are created on MR open and destroyed on MR merge.

- Define a review job with environment: name: review/$CI_COMMIT_REF_SLUG and on_stop: stop-review
- GitLab creates a dynamic environment per MR with a unique URL
- Deploy to a namespace or subdomain based on $CI_COMMIT_REF_SLUG
- stop-review job: runs on MR close/merge; tears down the environment
- Review App URL is linked in the MR widget — reviewers click to test directly

- Combine with Helm: helm upgrade --install $CI_COMMIT_REF_SLUG ./chart -n review-apps

Q 046

How do you configure GitLab CI/CD pipelines with includes for large, modular pipeline definitions?

▶ ANSWER

GitLab include lets you split a large .gitlab-ci.yml into multiple files and share common job templates across projects — solving the copy-paste pipeline problem.

- include:local — include a file from the same repository
- include:project — include a file from another GitLab project (e.g., a shared platform pipeline)
- include:remote — include a raw file from a URL
- include:template — use GitLab's built-in CI/CD templates (SAST, DAST, Container Scanning)
- extends: — reuse a hidden job template (defined with . prefix) within the same file
- !reference: — reuse arrays of steps across multiple jobs without duplication

Example

yaml / shell

```yaml
# .gitlab-ci.yml
include:
  - project: 'platform/ci-templates'
    ref: main
    file: '/templates/docker-build.yml'
  - template: Security/Container-Scanning.gitlab-ci.yml

stages: [build, scan, deploy]

.deploy-template: &deploy-template
  image: bitnami/kubectl:latest
  script:
    - kubectl apply -f k8s/ -n $KUBE_NAMESPACE

deploy-staging:
  <<: *deploy-template
  environment: staging
  variables:
    KUBE_NAMESPACE: staging
```

CH 08

Jenkins Pipelines

Q 047

What is a Jenkins Declarative Pipeline and how does it differ from Scripted?

▶ ANSWER

Declarative Pipeline provides a structured, opinionated, validated syntax with built-in constructs. Scripted Pipeline is pure Groovy with maximum flexibility but no enforced structure.

- Declarative: pipeline {} block enforces structure: agent, stages, steps, post
- Declarative: syntax validated before any step executes — typos fail immediately
- Declarative: built-in tools, credentials binding, options, and triggers
- Scripted: full Groovy DSL — loops, closures, try/catch anywhere in the file
- Recommended: use Declarative for all new pipelines; embed script {} for complex logic
- Both store pipeline definition in a Jenkinsfile committed to the repository root

Example

yaml / shell

```
pipeline {
  agent any
  options {
    timeout(time: 30, unit: 'MINUTES')
    buildDiscarder(logRotator(numToKeepStr: '10'))
  }
  environment {
    REGISTRY = 'ghcr.io/devopsshack'
    APP = 'myapp'
  }
  stages {
    stage('Build') {
      steps { sh 'docker build -t $REGISTRY/$APP:$BUILD_NUMBER .' }
    }
    stage('Deploy') {
      when { branch 'main' }
      steps { sh 'kubectl apply -f k8s/' }
    }
  }
```

```
  post {
    success { echo 'Pipeline passed' }
    failure { mail to: 'team@devopsshack.com', subject: 'Build Failed:
$JOB_NAME' }
    always  { cleanWs() }
  }
}
```

Q 048

How do you manage credentials securely in Jenkins pipelines and integrate with HashiCorp Vault?

▶ ANSWER

Jenkins Credentials Plugin stores secrets encrypted. withCredentials binding injects them into steps masked from logs. For dynamic credentials, the Vault plugin is the gold standard.

- Store at: Manage Jenkins → Credentials → (Global or folder-scoped)
- Types: Username/password, Secret text, SSH key, Certificate, Secret file
- withCredentials{} block: credentials injected as env vars, masked in all logs
- Never use env.PASS = credentials('id') — this exposes secrets as plain env vars
- Vault Plugin: issues dynamic, short-lived database credentials per build
- AppRole auth: Jenkins uses Vault AppRole to authenticate and retrieve secrets

Example

yaml / shell

```
stage('Build and Push') {
  steps {
    withCredentials([
      usernamePassword(
        credentialsId: 'ghcr-creds',
        usernameVariable: 'DOCKER_USER',
        passwordVariable: 'DOCKER_PASS'
      ),
      string(credentialsId: 'sonar-token', variable: 'SONAR_TOKEN')
    ]) {
      sh '''
        echo $DOCKER_PASS | docker login ghcr.io -u $DOCKER_USER --password-
stdin
        docker build -t ghcr.io/myorg/app:$BUILD_NUMBER .
        docker push ghcr.io/myorg/app:$BUILD_NUMBER
```

```
        '''
      }
    }
  }
```

Q 049

How do you scale Jenkins to handle 50+ parallel builds without agents queuing for 20+ minutes?

▶ ANSWER

The solution is dynamic, ephemeral agents — one pod per build using the Kubernetes plugin, rather than a fixed pool of long-running VMs.

- Jenkins Kubernetes Plugin: provisions a fresh pod for each build, destroys it after completion
- No queue: pods scale to cluster capacity; no fixed pool bottleneck
- Ephemeral security: each build runs in a clean pod with no state from previous builds
- Pod template: define agent image, resources, and volumes in pipeline or global config
- Faster startup: pre-pulled Docker images start in 10–30 seconds vs 5+ minutes for VMs
- Cost efficient: pods only consume resources during the build — no idle agent cost

Example

yaml / shell

```
pipeline {
  agent {
    kubernetes {
      yaml """
apiVersion: v1
kind: Pod
spec:
  containers:
  - name: docker
    image: docker:24-dind
    securityContext:
      privileged: true
  - name: kubectl
    image: bitnami/kubectl:latest
    command: [sleep]
    args: [infinity]
      """
    }
```

```
    }
}
```

Q 050

How do you implement automated test reporting and trend tracking in Jenkins?

▶ ANSWER

Jenkins has built-in JUnit test result ingestion. Publishing test results as post-build actions surfaces trends, detects flaky tests, and blocks builds when failure rate spikes.

- junit '**/target/surefire-reports/*.xml': publishes Maven Surefire results to Jenkins UI
- Build trend graph: Jenkins tracks pass/fail/skip counts across builds automatically
- Cobertura or JaCoCo plugin: publishes code coverage trends alongside test results
- Fail the build: set minimum coverage threshold — build fails if coverage drops below 80%
- Allure plugin: rich HTML test reports with screenshots, steps, and failure history
- testResultsSummary: check needs: false for non-blocking but recorded test stages

Example

yaml / shell

```
post {
  always {
    junit '**/target/surefire-reports/*.xml'
    jacoco(
      execPattern: '**/target/*.exec',
      minimumLineCoverage: '80'
    )
    publishHTML([
      reportDir: 'target/site',
      reportFiles: 'index.html',
      reportName: 'Test Report'
    ])
  }
}
```

CH 09

ArgoCD & GitOps Deployments

Q 051

How does ArgoCD work and what deployment problem does it solve?

▶ ANSWER

ArgoCD is a declarative GitOps CD operator for Kubernetes. It continuously reconciles the live cluster state to match the desired state defined in a Git repository.

- Runs inside the cluster as Kubernetes controllers — no external CD server needed
- Watches a Git repo (polling or webhook) for changes to manifests, Helm charts, or Kustomize overlays
- Detects drift: live state differs from Git desired state → shows diff in UI
- Syncs automatically (Continuous Deployment) or waits for approval (Continuous Delivery)
- UI and CLI show resource health, sync status, events, and complete deployment history
- Solves: manual kubectl apply, environment drift, no audit trail, no rollback history

Q 052

What is the difference between ArgoCD manual and automated sync? When do you use each?

▶ ANSWER

Sync strategy determines whether ArgoCD applies detected Git changes automatically or waits for explicit human action. Choose based on risk tolerance of the target environment.

- Manual sync: ArgoCD detects drift and waits — user clicks Sync in UI or runs argocd app sync
- Automated sync: ArgoCD applies every new commit to the target branch immediately
- selfHeal: true — automatically reverts any out-of-band kubectl change that diverges from Git
- prune: true — deletes K8s resources that no longer exist in the Git manifests
- Production recommendation: automated + selfHeal + prune for full consistency
- Use manual sync for production when compliance requires change approval before deployment

Q 053 How do you integrate ArgoCD with GitHub Actions CI? Walk through the complete flow.

▶ ANSWER

CI and ArgoCD have a clean separation of concerns: CI builds and pushes the artifact; ArgoCD detects the manifest change and deploys it.

- CI Step 1: run tests, build Docker image, push to registry with git SHA tag
- CI Step 2: update image tag in Kubernetes manifest YAML using yq or sed
- CI Step 3: commit the updated manifest and push to the GitOps repository (may be same repo)
- ArgoCD Step 1: detects the new commit via webhook or 3-minute polling
- ArgoCD Step 2: computes diff between current cluster state and new Git state
- ArgoCD Step 3: syncs cluster — replaces old pods with new image version

Example

yaml / shell

```
- name: Update manifest image tag
  run: |
    yq e '.spec.template.spec.containers[0].image = "ghcr.io/myorg/app:${{
github.sha }}"' \
      -i release/deployment.yaml


- name: Commit updated manifest
  run: |
    git config user.name 'github-actions'
    git config user.email 'actions@github.com'
    git pull --rebase origin main
    git add release/deployment.yaml
    git commit -m 'ci: update app image to ${{ github.sha }}' || true
    git push origin main


# ArgoCD detects the commit automatically (or trigger manually):
# argocd app sync myapp --server $ARGOCD_SERVER
```

Q 054 How do you manage multiple environments (dev, staging, prod) with a single ArgoCD installation?

▶ ANSWER

Use ApplicationSets or separate Application resources with different target namespaces, clusters, and source paths. Kustomize overlays or Helm values files provide per-environment configuration.

- Separate ArgoCD Application per environment: myapp-dev, myapp-staging, myapp-prod
- Each Application points to a different path: overlays/dev, overlays/staging, overlays/prod
- Kustomize: base/ contains common manifests; overlays/ patch environment-specific values
- Helm: separate values-dev.yaml, values-staging.yaml, values-prod.yaml files
- ApplicationSet: generates multiple Applications from a template + cluster/env list
- Production Application uses manual sync; staging and dev use automated sync

---

Q 055

How do you use Helm with ArgoCD for multi-environment deployments?

▶ ANSWER

Helm provides templating and values-based configuration. ArgoCD drives the deployment from Git, using different Helm values files per environment to customise each deployment.

- ArgoCD Application: set chart path and valueFiles pointing to values-staging.yaml or values-prod.yaml
- values-base.yaml: shared defaults (image repository, replica count, resource limits)
- values-staging.yaml: overrides for staging (replicas: 1, DEBUG: true, small resource limits)
- values-prod.yaml: overrides for production (replicas: 3, DEBUG: false, production resource limits)
- Helm secrets plugin: encrypt sensitive values in values files using SOPS or Vault
- ArgoCD app-of-apps: a parent Application manages child Applications for each environment

Example

yaml / shell

```yaml
# ArgoCD Application for staging
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: myapp-staging
spec:
  source:
    repoURL: https://github.com/myorg/myapp
    path: helm/myapp
    helm:
```

```
      valueFiles:
        - values-base.yaml
        - values-staging.yaml
  destination:
    server: https://kubernetes.default.svc
    namespace: staging
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

CH 10

Terraform & IaC in CI/CD

Q 056

How do you implement a safe Terraform CI/CD pipeline with plan review and apply gates?

▶ ANSWER

A Terraform pipeline exposes the plan for review on every PR and restricts apply to post-merge with explicit approval — preventing surprise infrastructure changes.

- terraform fmt --check: enforce consistent code formatting as the first step
- terraform validate: syntax and provider logic check with no cloud API calls
- terraform plan -out=tfplan: generate execution plan, save as artifact
- Post plan as PR comment: team reviews what will change before approving the PR
- Manual approval gate (GitHub Environment): reviewer approves terraform apply
- terraform apply tfplan: applies exactly the saved plan — not a new plan that might differ

Example

yaml / shell

```yaml
jobs:
  plan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${{ secrets.TF_ROLE_ARN }}
          aws-region: ap-south-1
      - run: terraform init
      - run: terraform validate
      - run: terraform fmt --check
      - run: terraform plan -out=tfplan
      - uses: actions/upload-artifact@v4
        with: { name: tfplan, path: tfplan }

  apply:
```

```
    needs: plan
    environment: production  # requires manual approval
    runs-on: ubuntu-latest
    steps:
      - uses: actions/download-artifact@v4
        with: { name: tfplan }
      - run: terraform apply -auto-approve tfplan
```

Q 057

How do you manage Terraform state safely in a team environment within CI/CD?

▶ ANSWER

Remote state with locking prevents concurrent apply conflicts. Every team member and pipeline run reads and writes the same authoritative state file.

- S3 backend: store state in S3 with versioning enabled for recovery and history
- DynamoDB lock table: prevents simultaneous applies from corrupting state
- State encryption: enable SSE-S3 or SSE-KMS on the S3 bucket
- Terraform workspaces: separate state files per environment (dev, staging, prod)
- Never commit .tfstate to Git — add *.tfstate and *.tfstate.backup to .gitignore
- RBAC: CI/CD role can read + write state bucket; developers can read-only for plan

Example

yaml / shell

```
terraform {
  backend "s3" {
    bucket         = "devopsshack-terraform-state"
    key            = "microservices/eks/terraform.tfstate"
    region         = "ap-south-1"
    encrypt        = true
    dynamodb_table = "terraform-lock"
  }
}
```

Q 058

A terraform apply failed halfway through. How do you safely recover without duplicating resources?

▶ ANSWER

Terraform is idempotent by design. A partial apply leaves state as-is for completed resources. Run plan to see the delta, fix the root cause, then re-apply.

- terraform plan: shows the exact diff between current state and desired config post-failure
- terraform state list: shows what was successfully provisioned before the failure
- Fix root cause first: quota exhaustion, IAM permission missing, provider bug, network issue
- terraform apply again: Terraform skips already-created resources and only creates missing ones
- Orphan cleanup: terraform destroy -target=<resource_type.name> for dangling partial resources
- State recovery: if state file is corrupted, restore a previous version from S3 versioned bucket

---

Q 059

How do you detect and handle Terraform state drift caused by manual changes made outside Terraform?

▶ ANSWER

State drift is when the real infrastructure diverges from the Terraform state file due to manual changes via console or CLI. Detect and resolve before the next apply.

- terraform plan shows drift: any manually changed resource appears as a 'change' to revert
- Scheduled drift detection: run terraform plan on a cron schedule; alert if output != 'No changes'
- terraform refresh (deprecated) or terraform apply -refresh-only: updates state to match reality
- Decide policy: import the manual change into Terraform, or let the next apply revert it
- Prevent drift: enforce IaC-only changes via SCPs/IAM denying console resource modifications
- Drift monitoring tools: Terraform Cloud, Env0, Spacelift provide drift detection dashboards

CH 11

Monitoring & Observability Post-Deployment

Q 060

How do you integrate deployment events with APM tools to correlate deploys with performance changes?

▶ ANSWER

Deployment markers on APM dashboards visually correlate a performance spike or error rate increase with the exact deployment that caused it — dramatically reducing MTTR.

- Send a deployment event to Datadog/New Relic/Dynatrace at the end of the deploy job
- APM shows deployment markers as vertical lines on performance graphs
- Immediately correlate: P99 latency spike starts exactly at deployment X timestamp
- Automatic alert: configure monitor to alert if error rate rises >2x within 5 min of deploy
- Rollback trigger: if alert fires within 10 min of deploy, automatically trigger rollback
- DORA integration: deployment events feed Lead Time for Changes and Deployment Frequency metrics

Example

yaml / shell

```yaml
- name: Send deployment event to Datadog
  run: |
    curl -X POST "https://api.datadoghq.com/api/v1/events" \
      -H "DD-API-KEY: ${{ secrets.DD_API_KEY }}" \
      -H "Content-Type: application/json" \
      -d '{
        "title": "Deployment: ${{ matrix.service }}",
        "text": "SHA: ${{ github.sha }}",
        "tags": ["env:production", "service:${{ matrix.service }}"],
        "alert_type": "info"
      }'
```

Q 061

How do you implement automated health check and rollback after a Kubernetes deployment?

▶ ANSWER

Post-deployment health checking closes the loop between deploying and confirming success. Automated rollback minimises customer impact when a bad deployment slips through.

- Poll the application health endpoint in a retry loop after kubectl apply
- 10 attempts with 15 second sleep — 150 second maximum wait for slow-starting apps
- kubectl rollout undo if health check fails — reverts to the previous ReplicaSet instantly
- Slack notification: alert the team with service name, failed SHA, and link to failed job
- ArgoCD: use argocd app rollback for GitOps-managed apps
- Combine with kubectl rollout status to ensure pod-level health before app-level check

Example

yaml / shell

```
- name: Post-deploy health check with auto-rollback
  run: |
    URL=http://myapp.devopsshack.com/health
    for i in {1..10}; do
      STATUS=$(curl -s -o /dev/null -w "%{http_code}" $URL)
      if [ "$STATUS" -eq 200 ]; then
        echo "Health check passed (attempt $i)"
        exit 0
      fi
      echo "Attempt $i: HTTP $STATUS — retrying in 15s"
      sleep 15
    done
    echo 'Health check failed — initiating rollback'
    kubectl rollout undo deployment/myapp -n production
    exit 1
```

Q 062

How do you implement SLO-based deployment validation in a CI/CD pipeline?

▶ ANSWER

SLO-based validation checks that key error rate and latency service level objectives are still being met after a deployment, before declaring the deployment successful.

- Define SLOs: e.g., error rate < 0.5%, P99 latency < 400ms, availability > 99.9%
- After deployment: query Prometheus, Datadog, or CloudWatch for SLO metrics over 5 minutes
- Compare against pre-deploy baseline: flag regression if error rate increases > 2x

- Pipeline step fails if any SLO breaches threshold — triggers automatic rollback
- Burn rate alerts: fast burn of error budget triggers immediate rollback trigger
- Tools: Sloth, OpenSLO, Google's SLO Generator for standardised SLO definition and querying

Example

yaml / shell

```
- name: SLO validation post-deploy
  run: |
    ERROR_RATE=$(curl -s "$PROMETHEUS_URL/api/v1/query" \
      --data-urlencode \

'query=rate(http_requests_total{status=~"5.."}[5m])/rate(http_requests_total[5m])'
\
      | jq '.data.result[0].value[1]' -r)
    echo "Current error rate: $ERROR_RATE"
    python3 -c "
    rate = float('$ERROR_RATE')
    if rate > 0.005:
        print('SLO BREACH: error rate', rate)
        exit(1)
    "
```

Q 063

How do you set up a Slack notification system for CI/CD pipeline events?

▶ ANSWER

Slack webhook notifications provide the team with real-time pipeline status without needing to watch the CI UI. Well-structured messages include enough context to act immediately.

- Create a Slack incoming webhook URL and store as SLACK_WEBHOOK_URL secret
- Notify on: pipeline failure (always), deployment success (production only), rollback events
- Include in message: service name, environment, commit SHA, author, link to pipeline run
- Use Slack Block Kit for rich formatted messages with color-coded status
- if: always() ensures failure notifications are sent even if job fails
- Differentiate channels: #ci-alerts for failures, #deploys for production deployments

Example

yaml / shell

```
- name: Notify Slack on failure
```

```
    if: failure()
    run: |
      curl -X POST ${{ secrets.SLACK_WEBHOOK_URL }} \
        -H 'Content-type: application/json' \
        -d '{
          "text": ":red_circle: *Pipeline Failed*",
          "attachments": [{
            "color": "danger",
            "fields": [
              {"title": "Repo", "value": "${{ github.repository }}"},
              {"title": "Branch", "value": "${{ github.ref_name }}"},
              {"title": "SHA", "value": "${{ github.sha }}"},
              {"title": "Run", "value": "${{ github.server_url }}/${{
github.repository }}/actions/runs/${{ github.run_id }}"}
            ]
          }]
        }'
```

CH 12

Advanced & Tricky CI/CD Scenarios

Q 064

Two developers push to main simultaneously. Both pipelines try to update the same manifest. How do you prevent race conditions?

▶ ANSWER

Concurrent manifest updates cause git conflicts or silent overwrites. Use pipeline concurrency groups and git pull --rebase to serialise updates.

- GitHub Actions concurrency: group per branch — queues runs instead of running in parallel
- cancel-in-progress: false — queue the next run; never cancel an in-flight deployment
- git pull --rebase origin main before every manifest commit to incorporate concurrent pushes
- Retry wrapper around git push for the rare simultaneous push edge case
- ArgoCD Image Updater: eliminates this problem entirely by handling manifest updates automatically
- Long-term: GitOps operator handles manifest updates; CI only builds and pushes images

Example

yaml / shell

```yaml
concurrency:
  group: manifest-update-${{ github.ref }}
  cancel-in-progress: false  # queue, don't cancel

- name: Commit and push manifest
  run: |
    git config user.name 'github-actions'
    git config user.email 'actions@github.com'
    git pull --rebase origin main
    git add release/kubernetes-manifests.yaml
    git commit -m "ci: update images ${{ github.sha }}" || true
    git push origin main
```

Q 065

A critical CVE is announced in a base image used by all 12 microservices. How do you patch all of them quickly?

▶ ANSWER

Patch base image CVEs at scale using automation — not by manually updating 12 Dockerfiles one by one.

- Shared internal base image: maintain one hardened base image; bump once, all services rebuild automatically
- Renovate Bot / Dependabot: auto-creates PRs to bump base image tags across all Dockerfiles in the repo
- workflow_dispatch trigger: dispatch a 'force rebuild all' workflow that rebuilds every service with the latest base
- Trivy in pipeline: even without explicit bump, the next natural build catches the CVE in image scan
- SBOM dashboard: immediately see which services use the vulnerable base image version
- SLA: document response times — CRITICAL CVE patched within 48 hours; HIGH within 7 days

---

Q 066

How do you prevent a race condition where update-manifest and deploy-to-k8s use different versions of the manifest?

▶ ANSWER

The manifest update job must complete and push before the deployment job checks out the repo. Use needs: dependency and fetch-depth: 0 with explicit ref: main checkout.

- deploy-to-k8s must have needs: update-manifest — guarantees manifest is pushed first
- Checkout with ref: main and fetch-depth: 0 — always gets the absolute latest commit
- Avoid using github.sha as the deployment checkout ref — that SHA may predate the manifest commit
- Add a git log --oneline -3 step to verify the manifest commit is present before applying
- In ArgoCD model: eliminate this entirely — CI pushes manifest, ArgoCD syncs independently

Example

yaml / shell

```yaml
deploy-to-k8s:
  needs: update-manifest
  steps:
    - uses: actions/checkout@v4
      with:
        ref: main
        fetch-depth: 0

    - name: Verify manifest is updated
      run: |
        echo 'Last 3 commits:'
```

```
          git log --oneline -3
          echo 'Active images in manifest:'
          grep 'image:' release/kubernetes-manifests.yaml
```

Q 067

A developer accidentally merges a breaking PR. What CI/CD controls should have prevented this?

▶ ANSWER

Breaking merges reveal gaps in multiple layers of protection: code review, automated testing, and deployment gating.

- Branch protection rule: require at least 1 PR review from a team member other than the author
- Required status checks: CI pipeline must pass before merge is allowed — non-negotiable
- CODEOWNERS: automatically assign senior engineers to review changes in high-risk paths
- Dismiss stale reviews: if new commits are pushed after approval, re-approval is required
- Staging gate: production deploy only after staging deployment and integration tests pass
- Rollback runbook: incidents happen regardless — minimise MTTR with documented procedures

Q 068

How do you implement a self-service deployment portal for non-engineers using GitHub Actions workflow_dispatch?

▶ ANSWER

workflow_dispatch with input parameters creates a clickable UI in GitHub Actions, enabling product managers and operations teams to trigger deployments with guardrails.

- workflow_dispatch adds a Run Workflow button in the GitHub Actions UI
- Input types: choice (dropdown), string (text input), boolean (checkbox)
- Required fields validation: GitHub blocks the run if required inputs are missing
- Environment approval still enforced: even manual dispatches go through production approval gate
- Audit trail: every dispatch logged with triggering user identity, inputs, and timestamp
- Restrict with branch protection: only main branch can be selected for production deployments

Example

yaml / shell

```
on:
  workflow_dispatch:
    inputs:
      environment:
        description: 'Target environment'
        type: choice
        options: [staging, production]
        required: true
      service:
        description: 'Service name to deploy'
        type: string
        required: true
      image_tag:
        description: 'Image tag (SHA or version)'
        type: string
        required: true
```

Q 069

How do you handle long-running E2E tests in CI without blocking every pull request?

▶ ANSWER

Categorise tests by speed and scope. Run fast tests on every PR; run slow tests on merge to main or on a nightly schedule.

- Fast tier (< 3 min): unit tests + linting — run on every PR commit, required for merge
- Medium tier (5–15 min): integration tests — run on PR to main, required for merge
- Slow tier (30+ min): E2E, load tests — run only on main merge or nightly cron
- Parallelise: split slow test suites across matrix jobs to run sub-suites concurrently
- Test caching: cache test fixtures and compiled test binaries between runs
- Flaky quarantine: move unstable tests to a non-blocking job labelled 'flaky' until fixed

Q 070

How do you implement a 'change freeze' period in a CI/CD pipeline without disabling automation entirely?

▶ ANSWER

Change freeze periods (during holidays, major events, audit periods) can be enforced at the pipeline level without removing automation — using environment protection rules or scheduled gates.

- GitHub Environments: add a wait timer or remove reviewers during the freeze period
- Branch protection: temporarily require additional reviewers for production deploys
- Deployment window check: add a step that fails if current time is outside approved deploy window
- Emergency bypass: document a break-glass procedure for critical hotfixes during freeze
- Auto-freeze calendar: integrate with PagerDuty or OpsGenie schedule to auto-enable freeze periods
- Communication: announce freeze in Slack and in the deployment pipeline description

Example

yaml / shell

```
- name: Check deployment window
  run: |
    HOUR=$(date -u +%H)
    DAY=$(date -u +%u)  # 1=Mon, 7=Sun
    # Only deploy Mon-Fri 09:00-17:00 UTC
    if [ "$DAY" -gt 5 ] || [ "$HOUR" -lt 9 ] || [ "$HOUR" -ge 17 ]; then
      echo 'Outside deployment window. Use workflow_dispatch to override.'
      exit 1
    fi
    echo 'Within deployment window, proceeding'
```

CH 13

AWS Native CI/CD Services

Q 071

Describe the AWS CI/CD ecosystem and when to choose native services over GitHub Actions.

▶ ANSWER

AWS offers a fully managed native CI/CD stack. Choose based on compliance, VPC isolation, and data residency requirements.

- CodeCommit: Git hosting with IAM auth, VPC endpoints, and no internet exposure required
- CodeBuild: managed build environment; pay per build minute; no agents to manage
- CodeDeploy: automates deployment to EC2, ECS tasks, Lambda, and on-premises instances
- CodePipeline: orchestrates the full source → build → test → deploy pipeline with approvals
- Choose AWS native when: all workloads on AWS, data must not leave AWS network, SOC2/HIPAA VPC isolation required
- Choose GitHub Actions when: multi-cloud, open-source ecosystem, modern DX, community actions needed

Q 072

How does AWS CodeDeploy implement blue/green deployment for ECS services?

▶ ANSWER

CodeDeploy creates a new ECS task set alongside the existing one, shifts ALB traffic progressively via two target groups, then terminates the old set.

- Two ALB target groups: Blue (current, live traffic) and Green (new, receives shifted traffic)
- appspec.yml defines hooks: BeforeInstall, AfterInstall, AfterAllowTestTraffic, BeforeAllowTraffic
- Traffic shifting strategies: Canary (10% then 100%), Linear (10% per minute), AllAtOnce
- CloudWatch alarm rollback: if a specified alarm triggers during traffic shift, CodeDeploy reverts automatically
- Test hook: run automated tests against green while it still receives 0% prod traffic
- Deployment group configuration links CodeDeploy, ECS cluster, service, and target groups

Q 073

How do you configure IAM permissions for a CodeBuild project that builds, scans, and pushes to ECR?

▶ ANSWER

CodeBuild assumes a service role. The role needs precisely scoped ECR permissions at both account level (GetAuthorizationToken) and repository level (push permissions).

- ecr:GetAuthorizationToken — account-level permission to get a Docker login token
- ecr:BatchCheckLayerAvailability — check if layers already exist before upload
- ecr:InitiateLayerUpload, ecr:UploadLayerPart, ecr:CompleteLayerUpload — push layers
- ecr:PutImage — write the final image manifest
- ecr:CreateRepository — optional, for auto-creating repos that do not yet exist
- Use the AWS-managed AmazonEC2ContainerRegistryPowerUser policy as a starting point, then restrict

Q 074

How do you build a complete end-to-end CI/CD pipeline using AWS CodePipeline, CodeBuild, and ECR?

▶ ANSWER

An AWS-native pipeline connects CodeCommit (source), CodeBuild (build + test), ECR (registry), and CodeDeploy (deploy) through CodePipeline as the orchestrator.

- Source stage: CodePipeline watches CodeCommit or GitHub for new commits
- Build stage: CodeBuild runs buildspec.yml — install, pre_build (ECR login), build (docker build), post_build (docker push)
- Test stage: separate CodeBuild project runs unit and integration tests
- Deploy stage: CodeDeploy deploys to ECS with blue/green or EC2 with rolling strategy
- Approval action: manual approval step before production deploy — sends SNS email to approvers
- Artifacts: CodePipeline passes imageDefinitions.json between stages to convey the new image tag

Example

yaml / shell

```
# buildspec.yml for CodeBuild
version: 0.2
phases:
  pre_build:
```

```
    commands:
      - aws ecr get-login-password | docker login --username AWS --password-
stdin $ECR_REGISTRY
      -
IMAGE=$ECR_REGISTRY/$IMAGE_REPO_NAME:$CODEBUILD_RESOLVED_SOURCE_VERSION
  build:
    commands:
      - docker build -t $IMAGE .
  post_build:
    commands:
      - docker push $IMAGE
      - printf '[{"name":"app","imageUri":"%s"}]' $IMAGE >
imagedefinitions.json
artifacts:
  files: imagedefinitions.json
```

Q 075

How do you implement AWS EKS cluster upgrades through a CI/CD pipeline with zero downtime?

▶ ANSWER

EKS upgrades require a careful sequence: upgrade control plane first, then managed node groups, then validate workloads. Automate the steps but gate each phase with health verification.

- Pre-upgrade: run kube-no-trouble (kubent) to detect deprecated API versions in current manifests
- Update control plane: aws eks update-cluster-version — takes 10–15 min; non-disruptive to workloads
- Verify control plane: wait for cluster status ACTIVE before proceeding
- Update node groups: aws eks update-nodegroup-version with maxUnavailable: 1 for rolling replacement
- Verify pods: kubectl get pods -A — ensure no pods are in non-Running state after node upgrade
- Post-upgrade: run integration tests to confirm workloads are healthy on the new K8s version

Example

yaml / shell

```
- name: Check deprecated APIs before upgrade
  run: |
    docker run --rm \
      -v $HOME/.kube:/root/.kube \
      ghcr.io/doitintl/kube-no-trouble:0 \
      -o wide
```

```
  - name: Upgrade EKS control plane
    run: |
      aws eks update-cluster-version \
        --name my-cluster \
        --kubernetes-version 1.31

  - name: Wait for control plane upgrade
    run: |
      aws eks wait cluster-active --name my-cluster
```

CH 14

Pipeline Reliability & Performance

Q 076

What are DORA metrics and how do you measure CI/CD pipeline health with them?

▶ ANSWER

DORA metrics are four research-backed indicators proven to correlate with both software delivery performance and organisational outcomes.

- Deployment Frequency: how often you deploy to production — elite = multiple times per day
- Lead Time for Changes: commit to production — elite = less than one hour
- Change Failure Rate: percentage of deployments causing production incidents — elite = 0–5%
- Mean Time to Recovery: time to restore after a production failure — elite = less than one hour
- Measure from pipeline events: capture deploy timestamps, failure events, incident start/end
- Tools: LinearB, Sleuth, GitHub Insights, or a custom Grafana dashboard fed by pipeline webhooks

Q 077

How do you detect and systematically eliminate flaky tests from CI/CD pipelines?

▶ ANSWER

Flaky tests erode team trust in the entire pipeline. Treat flakiness as a bug with a dedicated detection, quarantine, and fix process.

- Detect: record test pass/fail over time; flag tests failing more than 3% without code changes
- Quarantine: move to a parallel non-blocking job — they run but do not gate merge
- Fix priority: flaky tests are bugs; add to next sprint backlog with priority
- Retry logic: retry a failed test once before marking it failed — surfaces rare flakiness without noise
- Root causes: timing assumptions (Thread.sleep), test ordering dependencies, shared mutable global state
- Tools: BuildPulse, Trunk Flaky Tests, Allure Reports for automatic flakiness detection

Q 078 When should you use self-hosted runners for GitHub Actions and how do you set them up securely?

▶ ANSWER

Self-hosted runners are VMs or containers you manage. Use them when GitHub-hosted runners cannot meet your security, hardware, or cost requirements.

- Use when: builds must access VPC-internal databases, APIs, or artifact repositories
- Use when: specialised hardware needed — GPU, ARM64, 32+ GB RAM workloads
- Use when: at scale, self-hosted runners are significantly cheaper than GitHub-hosted minutes
- Security: use ephemeral runners — one runner per job, destroyed after completion
- Never use persistent self-hosted runners for public repos — anyone can run code on them
- actions-runner-controller (ARC): Kubernetes operator that auto-scales runner pods on demand

Q 079 How do you implement pipeline concurrency controls to prevent deployment chaos in high-frequency teams?

▶ ANSWER

Without concurrency controls, multiple simultaneous deploys can race each other, deploy out-of-order, or apply inconsistent manifests.

- GitHub Actions concurrency groups: limit concurrent runs per branch or environment
- cancel-in-progress: true — cancel outdated PR builds when a newer commit is pushed (saves runner minutes)
- cancel-in-progress: false — queue deploy pipelines; never cancel an in-flight production deploy
- Separate groups for CI (can cancel) vs CD (must queue): use different group names
- Deployment locks: use a mutex pattern (create/delete a K8s ConfigMap) for multi-cluster deploys
- Serialise manifest updates: concurrency group on the manifest-update job prevents git conflicts

Example

yaml / shell

```yaml
# CI: cancel old PR builds when new commit arrives
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true
```

```
# CD: queue deploys — never cancel an in-flight production deploy
concurrency:
  group: deploy-production
  cancel-in-progress: false
```

Q 080

How do you implement cost optimisation for a CI/CD infrastructure running 500+ builds per day?

▶ ANSWER

At high build volume, CI/CD infrastructure cost becomes a significant line item. Systematic optimisation targets compute, cache, and selective execution.

- Change detection: build only affected services — reduces build count by 60–80% in monorepos
- Spot/Preemptible instances: use for non-critical CI workloads — 70% cost reduction vs on-demand
- Docker layer cache: saves 6–8 min per build — at 500 builds/day, that is 50–67 runner-hours saved
- Concurrency limits: prevent runaway parallel jobs from spiking costs unexpectedly
- Artifact retention: reduce default 90-day retention to 7 days for large test artifacts
- Runner right-sizing: match runner size to job needs — a linting job does not need 16 vCPU

Q 081

How do you implement pipeline observability — making the pipeline itself observable like a production system?

▶ ANSWER

Treating CI/CD pipelines as production systems means instrumenting them with metrics, dashboards, and alerts — not just reactive log review when something breaks.

- Emit pipeline duration metrics per job and workflow using webhook events or API polling
- Track: build success rate, mean build duration, queue wait time, cache hit rate
- Grafana dashboard: pipeline health at a glance — trend over time, p50/p95 durations
- Alert on: build success rate dropping below 85%, mean duration increasing 50% over baseline
- GitHub API: pull workflow run data and push to a metrics store (Prometheus pushgateway, Datadog)
- OpenTelemetry for CI: otel-cicd-observability libraries emit spans for each pipeline step

CH 15

Real-World Troubleshooting Scenarios

Q 082

kubectl apply succeeds in the pipeline but pods enter CrashLoopBackOff. How do you investigate?

▶ ANSWER

kubectl apply validates YAML syntax and applies it to the API server. CrashLoopBackOff is a runtime crash — diagnosed via pod logs and events, not the manifest.

- kubectl logs <pod> --previous: logs from the crashed container's last run — the crash reason is here
- kubectl describe pod <pod>: Events section shows OOMKilled, ImagePullBackOff, probe failures
- OOMKilled: container exceeded memory limit — increase resources.limits.memory
- Missing env var: application panics on startup without a required DATABASE_URL or SECRET_KEY
- ImagePullBackOff: wrong image tag, private registry auth not configured (missing imagePullSecrets)
- Pipeline fix: add a post-deploy readinessProbe check step; do not consider deploy successful until pods are ready

Example

yaml / shell

```
# Step 1: Get crash logs
kubectl logs <pod-name> -n project --previous

# Step 2: Inspect events
kubectl describe pod <pod-name> -n project

# Step 3: Check resource usage
kubectl top pods -n project

# Step 4: Check all namespace events sorted by time
kubectl get events -n project \
  --sort-by=.lastTimestamp | tail -20
```

Q 083 GitHub Actions reports 'Resource not accessible by integration'. How do you diagnose and fix it?

▶ ANSWER

This error means GITHUB_TOKEN does not have the required permission scope. Since 2023, GitHub defaults all token permissions to read-only.

- Creating releases → add contents: write
- Pushing to GHCR → add packages: write
- Creating/updating PRs → add pull-requests: write
- Uploading SARIF to Security tab → add security-events: write
- Permissions can be set at workflow level or job level; job level overrides workflow level
- Avoid permissions: write-all — grant only the specific scopes you need (principle of least privilege)

Example

yaml / shell

```yaml
# At workflow level (applies to all jobs)
permissions:
  contents: write
  packages: write
  security-events: write

# Or at individual job level (more granular)
jobs:
  release:
    permissions:
      contents: write
    steps:
      - uses: actions/create-release@v1
```

Q 084 Terraform plan in CI shows 100+ unexpected resource changes when only 1 was expected. How do you diagnose?

▶ ANSWER

Mass unexpected changes in terraform plan indicate state drift, provider version changes, or unintended code edits.

- Provider version bump: upgrading providers can change resource schemas, causing replace or update actions
- State drift: someone made manual changes via AWS Console or CLI outside of Terraform
- terraform apply -refresh-only: update state to reflect current real infrastructure without changing anything
- terraform plan -target=<resource>: scope investigation to the specific resource you intended to change
- Version constraints: pin providers in required_providers with ~> (pessimistic constraint) to prevent surprise upgrades
- Prevention: enforce a policy that all infrastructure changes go through Terraform — deny console modifications via SCP

Q 085

How do you immediately roll back a production deployment when something goes wrong?

▶ ANSWER

Know your rollback commands before you need them. Different deployment mechanisms have different fastest paths.

- kubectl: kubectl rollout undo deployment/<name> -n <namespace> — reverts to previous ReplicaSet immediately
- Helm: helm rollback <release> <revision> — use helm history <release> to find the revision number
- ArgoCD: argocd app rollback <app> or one-click rollback in ArgoCD UI
- GitOps: git revert HEAD && git push — ArgoCD syncs back automatically within minutes
- Verify rollback: kubectl rollout status + curl health endpoint to confirm the revert worked
- Post-incident: blameless post-mortem and a regression test added to prevent recurrence

---

Q 086

A PR merge pipeline passes all tests but the production deployment fails 2 hours later. Why might this happen?

▶ ANSWER

A time gap between test and deploy creates a window for state changes. The environment at deploy time differs from the environment when tests ran.

- External dependency changed: a downstream API changed its response format between test and deploy
- Infrastructure change: a network policy, security group rule, or IAM permission was modified in the window

- Production config differs from staging: a secret is wrong, a feature flag is different, or a DB schema lags
- Rate limiting: production has stricter quotas that staging does not encounter
- Time-sensitive logic: code behaves differently at certain times of day (business hours, cron triggers)
- Solution: reduce the gap between merge and deploy; ensure staging perfectly mirrors production configuration

Q 087

How do you debug a GitHub Actions workflow that is failing intermittently with no obvious error message?

▶ ANSWER

Intermittent failures (flaky pipelines) require structured investigation: enable debug logging, identify patterns in failure history, and add defensive measures.

- Enable debug logging: add secret ACTIONS_STEP_DEBUG=true — verbose per-step output in logs
- Enable runner diagnostics: ACTIONS_RUNNER_DEBUG=true for runner-level debug info
- Identify pattern: does it fail for specific services? On specific runner sizes? At specific times of day?
- Check runner health: GitHub-hosted runners can have transient network or disk issues
- Add explicit timeouts: prevent indefinite hangs caused by network calls that never return
- Idempotent steps: ensure every step is safely re-runnable (cleanup before retrying docker builds)

Example

yaml / shell

```
# Add to repo secrets for debug session:
# ACTIONS_STEP_DEBUG = true
# ACTIONS_RUNNER_DEBUG = true

# Add timeout to prevent hangs:
- name: Docker push
  timeout-minutes: 10
  run: docker push ghcr.io/myorg/app:${{ github.sha }}

# Retry transient failures:
- name: Push with retry
  uses: nick-fields/retry@v3
  with:
    timeout_minutes: 10
    max_attempts: 3
    command: docker push ghcr.io/myorg/app:${{ github.sha }}
```

Q 088

A deployment passed CI and staging but caused a 500ms increase in P99 latency in production. How do you respond?

▶ ANSWER

A latency regression discovered post-production-deploy requires rapid triage: rollback to restore SLO, then investigate root cause with APM data.

- Immediate: confirm the latency increase is real and sustained (not a transient spike) via APM
- Decision gate: if SLO is breaching (P99 > SLO threshold), rollback immediately without investigation
- Rollback: kubectl rollout undo or ArgoCD rollback — verify latency returns to baseline within 5 min
- Root cause: compare flame graphs / distributed traces between old and new deployment
- Common causes: N+1 database query, synchronous external call with no timeout, new middleware layer
- Fix: address root cause, add a performance regression test (P99 assertion) to the pipeline

CH 16

Expert-Level CI/CD Mastery

Q 089

Explain immutable infrastructure and how CI/CD naturally enforces it.

▶ ANSWER

Immutable infrastructure means running instances are never patched or modified after deployment — they are always replaced entirely with a freshly built artifact.

- No SSH patching in production: if a change is needed, build a new image and redeploy it
- Eliminates configuration drift: servers that have been manually patched differently break consistency
- Docker containers are inherently immutable: you rebuild, never exec in and modify
- Rollback = redeploy previous image: instant, reliable, tested — exactly as deployed before
- AMI baking with Packer: pre-build EC2 AMIs with all configuration for immutable EC2 infrastructure
- CI/CD naturally enforces immutability: every commit produces a fresh, versioned artifact from a clean build

Q 090

What is progressive delivery and how does it extend CI/CD beyond simple deployments?

▶ ANSWER

Progressive delivery decouples deployment from release. Code is deployed to production but exposed only to controlled user subsets — then expanded based on health metrics.

- Feature flags: toggle features for specific users without any redeployment
- Canary releases: 5% of real traffic to new version; promote if error rate and latency hold
- A/B testing: different versions running simultaneously for different user segments
- Dark launching: new feature processes real production traffic but results are discarded
- Argo Rollouts: automated canary promotion based on Prometheus metric thresholds
- CI/CD deploys the code; progressive delivery controls who sees the new behaviour and when

Q 091

How do you implement policy as code in CI/CD to enforce security compliance automatically?

▶ ANSWER

Policy as code uses tools like OPA (Open Policy Agent) to automatically enforce organisational rules in the pipeline — zero manual review required.

- OPA/Conftest: write Rego policies to validate Kubernetes manifests and Terraform plans
- Example policies: all containers must set CPU/memory limits; no :latest image tags allowed
- conftest test fails the pipeline if any manifest violates a policy — never reaches the cluster
- OPA Gatekeeper: enforce the same policies at K8s admission controller level as a second layer
- Policies stored in Git: versioned, reviewed via PR, and auditable like application code
- Security team owns policy repo; app teams can read and understand why their build failed

Example

yaml / shell

```
# Rego policy: deny latest tag
package main

deny[msg] {
  input.kind == "Deployment"
  container := input.spec.template.spec.containers[_]
  endswith(container.image, ":latest")
  msg := sprintf("Container %v uses :latest tag", [container.name])
}

# In pipeline:
- name: Validate manifests with Conftest
  run: |
    conftest test release/ --policy policies/
```

Q 092

How do you ensure CI/CD pipeline supply chain integrity — protecting the pipeline from attack?

▶ ANSWER

The pipeline itself is an attack surface. Pin dependencies to exact SHAs, sign artifacts with provenance attestation, and audit third-party actions before use.

- Pin action versions to commit SHA, not mutable tags — uses: actions/checkout@v4 can be hijacked
- SHA pinning: uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683 is immutable
- Dependabot for Actions: auto-creates PRs to update pinned SHAs when security fixes release
- SLSA (Supply-chain Levels for Software Artifacts): build provenance attestation framework
- GitHub artifact-attestation action: signs build provenance into GHCR image as an OCI attestation
- Restrict third-party actions: GitHub org policy can limit which action owners are allowed

Example

yaml / shell

```
# Bad: mutable tag can be changed to point to malicious code
uses: actions/checkout@v4

# Good: SHA is immutable — cannot be altered without changing the reference
uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683  # v4.2.2

# Sign build provenance
- uses: actions/attest-build-provenance@v1
  with:
    subject-name: ghcr.io/myorg/app
    subject-digest: sha256:${{ steps.build.outputs.digest }}
```

Q 093

How do you design a multi-cloud CI/CD strategy for deploying to both AWS EKS and GCP GKE simultaneously?

▶ ANSWER

Multi-cloud deployments require cloud-agnostic image storage, per-cloud OIDC authentication, and an independent health-verified rollout per cloud region.

- Single image registry (GHCR): same image SHA deployed to both clouds — no rebuild
- OIDC to both AWS (assume IAM role) and GCP (workload identity federation) from one job
- Matrix deployment: deploy job runs for [aws, gcp] clouds in parallel
- Per-cloud health check: verify each cluster independently before proceeding
- Rollback independence: can rollback one cloud without touching the other
- Global load balancer (Route53 / Cloud DNS): routes users to nearest healthy region

Q 094

You are given 3 months to take a team from zero CI/CD to full continuous deployment. Describe your roadmap.

▶ ANSWER

A 3-month transformation balances speed of adoption with team trust. Each phase must demonstrate value before the next begins.

- Month 1, Week 1: Git branching strategy, PR workflow, code review culture
- Month 1, Week 2-3: CI foundation — automated build and unit tests on every PR
- Month 1, Week 4: Containerise — Docker builds and push to registry on merge
- Month 2, Week 1-2: Staging CD — auto-deploy to staging; DevSecOps scans (Gitleaks, Trivy, Checkov)
- Month 2, Week 3-4: Integration tests + SonarQube quality gate in staging pipeline
- Month 3, Week 1-2: Production pipeline — approval gates, health checks, auto-rollback
- Month 3, Week 3-4: DORA metrics dashboard, incident runbooks, team training, retrospective

Pro Tip Month 1 is about changing habits, not tools. If developers do not trust the CI results, they will bypass it. Win trust by making CI fast, reliable, and demonstrably valuable before adding complexity.

Q 095

How do you implement a canary deployment with automatic metric-based promotion using Argo Rollouts?

▶ ANSWER

Argo Rollouts provides automated canary analysis by querying Prometheus metrics and making promotion or rollback decisions without human intervention.

- Argo Rollouts replaces Kubernetes Deployment with a Rollout custom resource
- AnalysisTemplate queries Prometheus: if error rate > 5% → fail; if < 1% → pass
- Traffic split: 10% canary, 90% stable via weighted services or Istio VirtualService
- Step-based promotion: 10% → pause 5 min → analysis → 50% → analysis → 100%
- On analysis failure: automatic rollback to stable, no human intervention needed
- Full audit log: every step, metric value, and decision recorded in Rollout status

Q 096

If you could only pick 5 CI/CD best practices to implement from day one, what would they be and why?

▶ ANSWER

These five practices form the non-negotiable foundation. Everything else — GitOps, progressive delivery, chaos engineering — builds on top of them.

- 1. Pipeline-as-code: every pipeline in Git — reviewable, reproducible, auditable, recoverable
- 2. Automated tests on every commit: fast, reliable feedback; broken code never reaches main
- 3. Immutable image tagging with git SHA: complete traceability from production to the exact commit
- 4. Branch protection + required CI status checks: no broken, unreviewed code ever merges
- 5. Post-deploy health check + automated rollback: the safety net for everything that slips through

Pro Tip These five compound over time. A team with all five reliably ships faster with less stress than a team with 50 tools and none of these fundamentals. Start simple, do it right, then add sophistication.

Q 097

How do you implement compliance and audit trail requirements (SOC 2, ISO 27001) using CI/CD tooling?

▶ ANSWER

CI/CD tooling naturally generates evidence for compliance requirements. The key is capturing, structuring, and retaining that evidence in a durable, tamper-evident way.

- Access control evidence: branch protection rules, required reviews, CODEOWNERS — exportable from GitHub API
- Change approval records: GitHub Environment approvals logged with approver, timestamp, and SHA
- Separation of duties: developer who writes code cannot approve their own deployment
- Vulnerability management: Trivy/Checkov scan results stored as pipeline artifacts with timestamps
- SBOM inventory: generated and archived on every production build for component tracking
- Tamper-evident logs: export pipeline logs to S3 with Object Lock for immutable retention

Q 098

How do you implement chaos engineering within a CI/CD pipeline to validate system resilience?

▶ ANSWER

Chaos engineering validates that your system recovers from failures. Integrate controlled failure injection into the staging pipeline after deployment to test resilience automatically.

- Run after successful staging deployment: chaos experiments test the deployed version
- Chaos Mesh or LitmusChaos: Kubernetes-native chaos injection via CRDs
- Experiment types: pod deletion, network latency injection, CPU stress, memory pressure
- Steady-state hypothesis: define what 'normal' looks like — error rate < 0.1%, P99 < 300ms
- If hypothesis violated after chaos injection: fail the pipeline and block production promotion
- Gradually expand scope: start with single pod kill, grow to node failure, then zone failure

Example

yaml / shell

```
- name: Run chaos experiment
  run: |
    # Apply a pod-kill chaos experiment
    kubectl apply -f - <<EOF
    apiVersion: chaos-mesh.org/v1alpha1
    kind: PodChaos
    metadata:
      name: pod-kill-test
      namespace: staging
    spec:
      action: pod-kill
      selector:
        namespaces: [staging]
        labelSelectors:
          app: frontend
      mode: one
      duration: 30s
    EOF
    sleep 60
    curl -f http://staging.myapp.com/health || exit 1
```

Q 099 How do you implement a GitFlow-based branching strategy with a CI/CD pipeline that supports hotfixes?

▶ ANSWER

GitFlow uses multiple long-lived branches for structured release management. CI/CD pipelines must handle feature, develop, release, hotfix, and main branches with different automation rules.

- develop branch: CI runs on every push — build, test, deploy to dev environment automatically
- feature/* branches: CI runs on PR open/update — build and test only, no deployment
- release/* branches: CI deploys to staging automatically; QA signs off before main merge
- main branch: CI deploys to production with approval gate on every merge
- hotfix/* branches: CI deploys to production directly after single approval — fast path
- Tags (v1.2.3): trigger production release pipeline; create GitHub release with CHANGELOG

Q 100 What is trunk-based development and how does it differ from GitFlow for CI/CD pipeline design?

▶ ANSWER

Trunk-based development merges all changes directly to a single main branch multiple times per day, enabling true continuous integration without merge queues or branch complexity.

- All developers integrate to main/trunk daily or multiple times per day
- Feature branches last less than one day — no long-lived divergent branches
- Feature flags replace long-lived branches for incomplete features in production
- CI pipeline must be fast (< 5 min) and reliable — developers are blocked until it passes
- No merge conflicts from weeks-old branches: the integration pain is distributed, not accumulated
- Enables true continuous deployment: merge to trunk = automatic deploy (with feature flags as safety)

Thank You for Reading

DevOps CI/CD Scenario-Based Interview Q&A

Found this useful? Share it with your team.

Subscribe to DevOps Shack for more content like this.

devopsshack.com

YouTube · Instagram @devopsshack · LinkedIn